Comprehensive Notes on Transformer Architecture

1 High-Level Transformer Architecture

A Transformer consists of stacked layers, each containing:

1. **Multi-Head Self-Attention** (MHSA): lets each token attend to other tokens.
2. **Feed-Forward Network** (FFN): applies nonlinear transformation to each token individually.
3. **Residual Connections**: stabilize gradient flow.
4. **Layer Normalization**: normalizes activations within each token.

The encoder uses only self-attention. The decoder uses masked self-attention plus cross-attention.

2 Dot-Product Attention vs Summation

Attention computes similarity between queries Q and keys K using dot products:

$$\text{attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V.$$

Reasons for using dot products instead of summation:

- Dot products measure **directional similarity** between vectors.
- They scale naturally with feature dimension and can be normalized.
- Dot products fit matrix multiplication operations, enabling efficient GPU/TPU acceleration.
- Summing vectors loses directional information and cannot represent similarity.

3 Why Transformers Use LayerNorm Instead of BatchNorm

BatchNorm normalizes across the batch dimension, but this is problematic because:

- Transformer sequence lengths vary, so batch statistics become unstable.
- Autoregressive (decoder) models cannot depend on future-token batch statistics.
- BatchNorm interacts poorly with residual connections and attention, causing training instability.

LayerNorm instead normalizes across **features within each token**, making it:

- independent of batch size,
- stable for variable-length sequences,
- effective for attention-heavy architectures.

4 What the Feed-Forward Network Learns

The FFN is applied identically to each token:

$$\mathrm{FFN}(x) = W_2\,\sigma(W_1 x + b_1) + b_2.$$

It learns:

- nonlinear mixing of features produced by attention,
- richer token representations by expanding dimensionality (typically $4d$),
- transformations that would be hard for linear attention layers alone.

The FFN accounts for most parameters in a Transformer layer.

5 How to Reduce FFN Parameters

Several techniques reduce FFN size:

- Decrease the intermediate hidden dimension d_{ff} (e.g., from $4d$ to $2d$).
- Use **SwiGLU** or **GEGLU** with a smaller expansion ratio.
- Use **low-rank factorization** of FFN weight matrices.
- Replace the full FFN with a **gated** or **bottleneck** alternative.

6 FFN Activations and Loss Functions

FFN typically uses the following activations:

- ReLU (original Transformer),
- GELU (BERT, GPT-2),
- SwiGLU (LLaMA, PaLM; current best practice).

Loss functions:

- The FFN itself does not have a loss function.
- Language models use **cross-entropy loss** applied after the final output projection.

7 Time and Space Complexity

Let n = sequence length, d = hidden size.

- **Self-attention**: $O(n^2 d)$ from computing all pairwise token interactions.
- **Encoder layer**: dominated by self-attention, $O(n^2)$ memory and compute.
- **FFN**: $O(ndd_{ff})$, often $O(nd^2)$.

Attention is the primary bottleneck when n is large.

8 Reducing Attention from $O(n^2)$ to $O(n)$

Several approaches reduce attention complexity:

- **Sparse attention**: restrict tokens to attend only to local neighbors or patterns (Longformer, BigBird).
- **Low-rank projection**: approximate attention using low-rank K, V matrices (Linformer).
- **Kernelized attention**: approximate the softmax kernel using random feature maps (Performer).
- **State space models**: replace attention entirely with linear recurrent layers (Mamba, S4).

These reduce cost to $O(n)$ or $O(n \log n)$ depending on method.

9 Wider vs Deeper Transformers

Increasing width:

- improves representational richness,
- helps store more factual and semantic knowledge.

Increasing depth:

- improves hierarchical reasoning,
- helps with compositionality and multi-step transformations.

Both improve performance, but scaling laws show depth generally provides more efficient gains.

10 Parallelization

10.1 Encoder Parallelization

The encoder is fully parallelizable because:

- all tokens attend to each other at once,
- there is no causal constraint.

10.2 Decoder Parallelization

During training, the decoder is also parallelizable:

- all tokens are known,
- a masking matrix enforces causality,
- all positions can compute their outputs simultaneously.

During inference:

- it is **not** parallelizable across time,
- each token depends on all previously generated tokens,
- generation is sequential.

11 Learning Rate Scheduling

Transformers do not use a constant learning rate. The original schedule is:

$$\text{lr} = d_{\text{model}}^{-1/2} \cdot \min\left(t^{-1/2},\ t \cdot \text{warmup}^{-3/2}\right).$$

Key phases:

- **Warmup**: LR increases linearly during early steps to stabilize gradients.
- **Decay**: LR decreases as $1/\sqrt{t}$ for training stability.

Modern alternatives:

- cosine decay + warmup,
- linear decay + warmup,
- constant LR for small models or fine-tuning.

Transformers typically use AdamW with:

$$\beta_1 = 0.9, \quad \beta_2 = 0.98 \text{ or } 0.999, \quad \text{weight decay} = 0.01.$$